

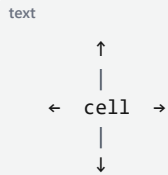
Memoization Approach and Code

This is a **classic 2D DP + memoization** problem. The key is to think in terms of:

```
dp(row, col, movesLeft) = number of ways to get out of the grid from this position using at most movesLeft moves.
```

1. The important observation

At every cell, the ball has 4 choices:



For each move:

- If the next position is **outside the grid** → we found **1 valid path**.
- If the next position is **inside** → recursively calculate the number of ways from there.
- If `movesLeft == 0` → we cannot move anymore, so answer is `0`.

So our recursive state is:

```
C++
solve(row, col, movesLeft)
```

2. Recurrence

Suppose we're at (r, c) .

There are four possible next positions:

```
C++
(r-1, c) // up
(r+1, c) // down
(r, c-1) // left
(r, c+1) // right
```

For each:

```
text
if outside:
    +1
else:
    + solve(nextRow, nextCol, movesLeft - 1)
```

Therefore:

```
text
dp(r, c, moves) =
    dp(up) +
    dp(down) +
    dp(left) +
    dp(right)
```

with the special case that going outside contributes `1`.

3. Why memoization?

Without memoization, the same states get calculated again and again.

For example:



Many different paths eventually reach the same:

```
text
(row, col, movesLeft)
```

So instead of recalculating it, store the answer.

Our DP table is:

```
C++
dp[50][50][51]
```

Meaning:

```
text
dp[r][c][k]
```

= number of ways to escape from (r, c) with k moves remaining.

4. Base cases

Case 1: We are already outside

We actually don't need to call recursion for an outside position.

When making a move:

```
C++
if (nr < 0 || nr >= m || nc < 0 || nc >= n)
    ans += 1;
```

That means this particular move successfully escaped.

Case 2: No moves remaining

```
C++
if (moves == 0)
    return 0;
```

Because we cannot make another move.

5. Memoization code

```
C++
class Solution {
public:
    const int MOD = 1e9 + 7;
    int m, n;
    int dp[50][50][51];

    int solve(int r, int c, int moves) {
        if (moves == 0)
            return 0;

        if (dp[r][c][moves] != -1)
            return dp[r][c][moves];

        long long ans = 0;

        int dr[] = {-1, 1, 0, 0};
        int dc[] = {0, 0, -1, 1};

        for (int i = 0; i < 4; i++) {
            int nr = r + dr[i];
            int nc = c + dc[i];

            // Ball goes outside the grid
            if (nr < 0 || nr >= m || nc < 0 || nc >= n) {
                ans++;
            }
            // Ball remains inside
            else {
                ans += solve(nr, nc, moves - 1);
            }
        }

        ans %= MOD;

        return dp[r][c][moves] = ans;
    }
}
```

```

int findPaths(int m, int n, int maxMove, int startRow, int startColumn) {
    this->m = m;
    this->n = n;

    memset(dp, -1, sizeof(dp));

    return solve(startRow, startColumn, maxMove);
}
};

```

6. Let's understand the recursion with Example 1

text

```

m = 2
n = 2
maxMove = 2
start = (0,0)

```

Grid:

text

```

      c0    c1
+-----+-----+
r0 |  S  |      |
+-----+-----+
r1 |      |      |
+-----+-----+

```

We start:

text

```

solve(0,0,2)

```

From (0,0) there are 4 directions.

Move UP

text

```

(-1,0)

```

Outside.

Therefore:

text

```

+1

```

Move LEFT

text

```

(0,-1)

```

Outside.

Therefore:

text

```

+1

```

Move RIGHT

text

```

(0,1)

```

Inside.

So:

text

```

solve(0,1,1)

```

Move DOWN

text

```

(1,0)

```

Inside.

So:

```
text
solve(1,0,1)
```

Therefore:

```
text
solve(0,0,2)
=
1 + 1 + solve(0,1,1) + solve(1,0,1)
```

7. Calculate solve(0,1,1)

We're here:

```
text
+-----+-----+
|         | X   |
+-----+-----+
|         |     |
+-----+-----+
```

Only 1 move remains.
Four directions:

UP

```
text
(-1,1)
```

Outside → 1

RIGHT

```
text
(0,2)
```

Outside → 1

LEFT

```
text
(0,0)
```

Inside, but after this move:

```
text
moves = 0
```

So:

```
text
solve(0,0,0) = 0
```

DOWN

```
text
(1,1)
```

Inside → 0

Therefore:

```
text
solve(0,1,1) = 1 + 1 + 0 + 0
              = 2
```

Similarly:

```
text
solve(1,0,1) = 2
```

Therefore:

```
text
solve(0,0,2)
= 1 + 1 + 2 + 2
= 6
```

Answer:

```
text
6
```

8. The most important DP idea here

Don't think:

"How many paths can the ball take?"

Think:

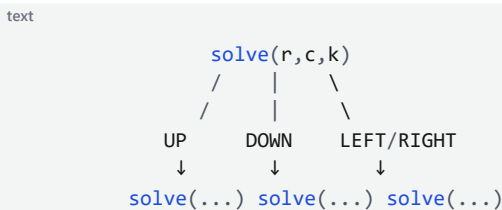
"From this state (row, col, movesLeft), how many ways can I escape?"

That's the entire problem.

The state has **3 dimensions**:

```
text
(row, column, moves)
```

and the transition has **4 choices**.



Complexity

There are at most:

```
text
m × n × maxMove
= 50 × 50 × 50
= 125,000 states
```

Each state checks 4 directions.

So:

```
text
Time: O(m × n × maxMove × 4)
      = O(m × n × maxMove)

Space: O(m × n × maxMove)
```

For the constraints, this is easily manageable.

One subtle point

The phrase "**at most** `maxMove` **moves**" doesn't require a separate calculation for every move count. A path is counted the moment it crosses the boundary, and it necessarily uses no more than the available moves. The recursion naturally handles this.

Yes. The tabulation version is basically the **same recurrence as memoization**, but instead of starting from the starting cell and recursively going forward, we build the DP table **from smaller number of moves to larger number of moves**.

The important part is understanding **why the direction of calculation changes**.

1. Start with our memoization recurrence

We had:

```
C++
solve(r, c, moves)
```

Meaning:

Number of ways to escape from (r,c) when I have `moves` moves available.

The recurrence was:

```
text
solve(r,c,moves)
=
solve(r-1,c,moves-1)
+
solve(r+1,c,moves-1)
+
solve(r,c-1,moves-1)
+
solve(r,c+1,moves-1)
```

But if a neighboring position is outside:

```
text
+1
```

And:

```
C++
solve(r,c,0) = 0
```

2. Convert this into a DP table

In memoization we had:

```
C++
dp[r][c][moves]
```

We can use exactly the same DP meaning in tabulation:

```
C++
dp[moves][r][c]
```

Let's define it as:

```
dp[k][r][c] = number of ways to escape from (r,c) using at most k moves.
```

So:

```
text
dp[0][r][c] = 0
```

because with zero moves, we cannot escape.

3. Now ask: how do we calculate `dp[1]`?

Suppose:

```
text
dp[1][r][c]
```

We have exactly one move available.

From (r,c) , check all four directions.

If the direction goes outside:

```
text
+1
```

If it remains inside:

```
text
dp[0][nr][nc]
```

But:

```
text
dp[0][nr][nc] = 0
```

So for $k = 1$, we can directly calculate every cell.

4. General transition

For every number of moves k :

text

```
dp[k][r][c]
```

look at the four neighboring cells.

For example, UP:

text

```
nr = r - 1  
nc = c
```

If outside:

C++

```
dp[k][r][c] += 1;
```

Otherwise:

C++

```
dp[k][r][c] += dp[k-1][nr][nc];
```

Same for all four directions.

Therefore:

text

```
dp[k][r][c] =  
    outside contribution  
    +  
    dp[k-1][up]  
    +  
    dp[k-1][down]  
    +  
    dp[k-1][left]  
    +  
    dp[k-1][right]
```

5. Tabulation code

C++

```
class Solution {  
public:  
    int findPaths(int m, int n, int maxMove, int startRow, int startColumn) {  
        const int MOD = 1e9 + 7;  
  
        vector<vector<vector<int>>> dp(  
            maxMove + 1,  
            vector<vector<int>>(m, vector<int>(n, 0))  
        );  
  
        int dr[] = {-1, 1, 0, 0};  
        int dc[] = {0, 0, -1, 1};  
  
        for (int moves = 1; moves <= maxMove; moves++) {  
            for (int r = 0; r < m; r++) {  
                for (int c = 0; c < n; c++) {  
                    long long ways = 0;  
  
                    for (int d = 0; d < 4; d++) {  
                        int nr = r + dr[d];  
                        int nc = c + dc[d];  
  
                        // Moving outside the grid  
                        if (nr < 0 || nr >= m || nc < 0 || nc >= n) {  
                            ways++;  
                        }  
                        // Moving inside the grid  
                        else {  
                            ways += dp[moves - 1][nr][nc];  
                        }  
                    }  
                    dp[moves][r][c] = ways % MOD;  
                }  
            }  
        }  
        return dp[maxMove][startRow][startColumn];  
    }  
};
```

```

        ways %= MOD;
    }

    dp[moves][r][c] = ways;
}
}

return dp[maxMove][startRow][startColumn];
};

```

6. How exactly did memoization become tabulation?

This is the most important thing to understand.

Memoization

We wrote:

```

C++
solve(r, c, moves)

```

and inside:

```

C++
solve(nr, nc, moves - 1)

```

So recursion goes:

```

text
moves
↓
moves - 1
↓
moves - 2
↓
...
0

```

For example:

```

text
solve(0,0,3)
↓
solve(0,1,2)
↓
solve(1,1,1)
↓
solve(1,0,0)

```

Tabulation

We simply reverse that process.

First calculate:

```

text
dp[0]

```

Then:

```

text
dp[1]

```

Then:

```

text
dp[2]

```

Then:

```

text
dp[3]

```

So:

text

```
dp[0] → dp[1] → dp[2] → dp[3]
```

This is exactly why the loop is:

C++

```
for (int moves = 1; moves <= maxMove; moves++)
```

7. Let's dry run a tiny example

Take:

text

```
m = 2
n = 2
maxMove = 2
```

Grid:

text

```
(0,0) (0,1)
(1,0) (1,1)
```

We initially have:

text

```
dp[0]
```

Every cell is 0:

text

```
0 0
0 0
```

Because with zero moves, nobody can escape.

Calculate dp[1]

Consider (0,0).

Possible moves:

text

```
      OUT
      ↑
OUT ← (0,0) → (0,1)
      ↓
      (1,0)
```

Two directions go outside:

text

```
UP    → 1
LEFT  → 1
```

Two directions stay inside:

text

```
RIGHT → dp[0][0][1] = 0
DOWN  → dp[0][1][0] = 0
```

Therefore:

text

```
dp[1][0][0] = 1 + 1 + 0 + 0
              = 2
```

Similarly every corner has two ways out:

text

```
dp[1]:
```

```
2 2
2 2
```

8. Now calculate `dp[2][0][0]`

We're interested in the starting position $(0,0)$.

Again:

UP

Outside:

```
text
+1
```

LEFT

Outside:

```
text
+1
```

RIGHT

Inside at $(0,1)$:

```
text
dp[1][0][1] = 2
```

DOWN

Inside at $(1,0)$:

```
text
dp[1][1][0] = 2
```

Therefore:

```
text
dp[2][0][0]
=
1 + 1 + 2 + 2
=
6
```

And the answer is:

```
text
6
```

9. Visualizing the whole tabulation

For this example:

`moves = 0`

```
text
0 0
0 0
```

`moves = 1`

```
text
2 2
2 2
```

`moves = 2`

```
text
6 6
6 6
```

Therefore:

```
text
answer = dp[2][0][0] = 6
```

10. The key conversion rule

Whenever you convert **memoization** → **tabulation**, follow this process:

Memoization:

```
C++  
solve(state)
```

has a dependency on:

```
C++  
solve(smaller_state)
```

Here:

```
C++  
solve(r,c,moves)
```

depends on:

```
C++  
solve(nr,nc,moves-1)
```

Therefore tabulation must calculate:

```
text  
  
moves = 0  
moves = 1  
moves = 2  
...  
moves = maxMove
```

So the recursive call:

```
C++  
solve(nr, nc, moves - 1)
```

becomes:

```
C++  
dp[moves - 1][nr][nc]
```

That's the direct conversion.

One more important optimization

We actually **don't need the entire 3D array**.

Notice:

```
C++  
dp[moves][r][c]
```

only depends on:

```
C++  
dp[moves - 1][nr][nc]
```

It doesn't depend on `dp[moves-2]`, `dp[moves-3]`, etc.

Therefore we can keep only two 2D arrays:

```
text  
  
previous  
current
```

and get:

```
text  
  
Space:  $O(m \times n)$ 
```

instead of:

```
text  
  
 $O(\text{maxMove} \times m \times n)$ 
```

That is the natural next optimization after understanding the 3D tabulation.

Exactly. Since `dp[moves][r][c]` only depends on `dp[moves-1][nr][nc]`, we **don't need to store all move levels**.

We only need:

text

previous → answers for moves - 1
current → answers for moves

Space-optimized tabulation

C++

```
class Solution {
public:
    int findPaths(int m, int n, int maxMove, int startRow, int startColumn) {
        const int MOD = 1e9 + 7;

        vector<vector<int>> prev(m, vector<int>(n, 0));
        vector<vector<int>> curr(m, vector<int>(n, 0));

        int dr[] = {-1, 1, 0, 0};
        int dc[] = {0, 0, -1, 1};

        for (int moves = 1; moves <= maxMove; moves++) {

            for (int r = 0; r < m; r++) {
                for (int c = 0; c < n; c++) {

                    long long ways = 0;

                    for (int d = 0; d < 4; d++) {
                        int nr = r + dr[d];
                        int nc = c + dc[d];

                        // Moving outside the grid
                        if (nr < 0 || nr >= m || nc < 0 || nc >= n) {
                            ways++;
                        }
                        // Moving inside the grid
                        else {
                            ways += prev[nr][nc];
                        }
                    }

                    ways %= MOD;

                    curr[r][c] = ways;
                }
            }

            // Current becomes previous for next iteration
            prev = curr;
        }

        return prev[startRow][startColumn];
    }
};
```

How the optimization works

Previously we had:

text

```
dp[0]
dp[1]
dp[2]
dp[3]
...
dp[maxMove]
```

That's a 3D array.

But look at the transition:

C++

```
dp[moves][r][c] = dp[moves - 1][neighbor]
```

For calculating `dp[5]`, do we need `dp[0]`, `dp[1]`, `dp[2]`, `dp[3]`?

No.

We only need:

```
text

dp[4] → calculate → dp[5]
```

So we rename:

```
text

dp[moves - 1] → prev
dp[moves]      → curr
```

Visually:

```
text

      previous
    +-----+
    | dp[k-1] |
    +-----+
      |
      | calculate
      ↓
    +-----+
    | dp[k]   |
    +-----+
      current
```

After finishing k :

```
text

curr becomes prev
```

Then:

```
text

prev = dp[k]
curr = dp[k+1]
```

and repeat.

One small improvement

The line:

```
C++

prev = curr;
```

copies the whole matrix every iteration. For these constraints (50×50) that's completely fine, but you can avoid that copy using `swap()` :

```
C++

swap(prev, curr);
```

Then the optimized version becomes:

```
C++

class Solution {
public:
    int findPaths(int m, int n, int maxMove, int startRow, int startColumn) {
        const int MOD = 1e9 + 7;

        vector<vector<int>> prev(m, vector<int>(n, 0));
        vector<vector<int>> curr(m, vector<int>(n, 0));

        int dr[] = {-1, 1, 0, 0};
        int dc[] = {0, 0, -1, 1};

        for (int moves = 1; moves <= maxMove; moves++) {

            for (int r = 0; r < m; r++) {
                for (int c = 0; c < n; c++) {

                    long long ways = 0;

                    for (int d = 0; d < 4; d++) {
                        int nr = r + dr[d];
                        int nc = c + dc[d];
```

```

        if (nr < 0 || nr >= m || nc < 0 || nc >= n)
            ways++;
        else
            ways += prev[nr][nc];

        ways %= MOD;
    }

    curr[r][c] = ways;
}

}

swap(prev, curr);
}

return prev[startRow][startColumn];
}
};

```

Complexity

text

Time:

$O(\text{maxMove} \times m \times n \times 4)$
 $= O(\text{maxMove} \times m \times n)$

Space:

$O(m \times n)$

The important DP progression to remember is:

text

Memoization

↓

$dp[r][c][moves]$

↓

Tabulation

↓

$dp[moves][r][c]$

↓

Notice only moves-1 is required

↓

Space optimization

↓

$prev[r][c] + curr[r][c]$

That's the exact reasoning you should use whenever you're asked to **space-optimize a DP**.